Master's thesis

E-graphs modulo theories

with applications to AC

Sofia Brookie
sofia.ma@protonmail.com

Supervisor: Nicholas Smallbone
Examiner: David Sands

May 6, 2026

Contents

1 Overview **3**
- 1.1 Introduction . . . 3
- 1.2 An introduction to E-graphs and equality saturation . . . 4
  - 1.2.1 Equational theorem proving . . . 4
  - 1.2.2 E-graphs for congruence-closure . . . 5
  - 1.2.3 E-graphs for equality saturation . . . 6
- 1.3 E-graphs and the problems of AC . . . 7
- 1.4 Handling AC with EMTs: a first example . . . 9
- 1.5 Challenges for EMTs . . . 10
  - 1.5.1 How E-graphs handle ordinary critical pairs, but not AC-critical pairs . . . 13
  - 1.5.2 Equality satuation: handling AC-critical pairs by brute force . . . 14
  - 1.5.3 EMTs: Handling AC-critical pairs less naïvely . . . 14

2 E-graphs in depth **16**
- 2.1 Equivalence relations and Union-find . . . 16
- 2.2 Congruence relations and Congruence-closure . . . 17
  - 2.2.1 Congruence closure builds a rewrite system . . . 21
- 2.3 Equality saturation . . . 21
  - 2.3.1 A few variations . . . 23

3 Which theories do E-graphs struggle with? **24**

4 E-graphs modulo theories **25**
- 4.1 EMTs: prior attempts . . . 26
  - 4.1.1 Intuition 1: Canonizers . . . 26
  - 4.1.2 Intutition 2: Containers . . . 26
  - 4.1.3 Canonizers and containers and the problem of flattening . . . 27
- 4.2 EMTs and the word problem . . . 30
  - 4.2.1 The problem of matching modulo theories . . . 30
- 4.3 E-graphs modulo theorie . . . 30

4.3.1 Defining EMTs 30

5 E-graphs modulo AC **32**
5.1 Solving the word problem modulo AC 32
5.1.1 Polynomial ideals and the AC embedding 32
5.1.2 Gröbner bases for ideals 33
5.1.3 Computing Grb̈ner bases via Buchberger's algorithm . . . 33
5.2 Matching modulo AC 33

6 An implementation **34**

7 Evaluation **35**

8 Prior work **36**

9 Conclusion and future work **37**

Chapter 1

Overview

1.1 Introduction

E-graphs are a data structure for equational theorem proving, and which provide the basis for *equality saturation* (section 2.3), a method to automatically derive equational consequences from a starting set of terms, equations, and identities. E-graphs have generated a lot of interest since the publication of the Egg library developed by Willsey et al. and their corresponding paper ([8]).

However, E-graphs struggle with a combinatorial explosion when reasoning about certain theories. A notorious case is that of AC theories, those containing associative and commutative operators: the space complexity of saturation is doubly-exponential in the size of the input in the worst case, as shown in section 1.3.

Can equality saturation for AC theories be done in a more efficient way? There have been several proposals for extending E-graphs to *E-graphs modulo theories* (EMTs) ([9], [3], [4]). These proposals suggest integrating a *theory-specific* reasoning method for AC together with the general mechanisms of E-graphs and equality saturation in order to efficiently deal with AC theories. The hope is that specific methods for AC theories can avoid the inefficiencies in using the general equality saturation mechanism for the AC identities.

EMTs are inspired by the success of theory-specific methods in other automated reasoning tasks, such as in *Satisfiability Modulo Theories* solvers (SMT solvers), and the merits of a theory-specific approach in automated reasoning are advocated for in Shankar [5]

We will develop the idea of EMTs and show that it *is* possible to reason with AC theories and a variety of closely related theories in a way that avoids the

worst inefficiencies of plain E-graphs.

In section 4.3, we also provide a formal definition of EMTs that we hope can provide the basis for future work on integrating other theories in a similar manner.

We then, in section 6, discuss an implementation of EMTs and benchmark it against pure E-graph reasoning on a variety of equational reasoning tasks (7).

Central to our notion of a 'good' theory for an EMT is the necessity of a solver for the word problem over the theory, which we justify in section 4.2. This gives us a lower bound on EMTs for AC: the word problem for AC requires in the worst case exponential space and doubly-exponential time. This is an improvement over the doubly-exponential space upper bound for E-graphs, but highlights that concrete efficiency gains derive from the potential for typical problems to be solvable efficiently. Our solver for AC and polynomials is builds upon Buchberger's algorithm, which has received significant effort to achieve good common-case performance.

On the other hand, for the theory ACUN (AC with unit and negation, aka the theory of Abelian groups) or the theory of vector spaces, we have polynomial time bounds: we can use algorithms for Hermite normal form (for Abelian groups) and Gaussian elimination (for vector spaces).

1.2 An introduction to E-graphs and equality saturation

1.2.1 Equational theorem proving

E-graphs are a data structure to handle *equational theorem proving*: we are given a set of *terms*, a set of *equations*, and a set of *identities*, and we try to determine if some equation follows from the conjunction of these equations and identities.

An equation between terms s and t will be written as $s = t$. An **identity** is a universally quantified equation: for instance, $\forall xy.x + y = y + x$ is an identity, asserting that the equation $x + y = y + x$ holds universally in x and y. To be more precise, an identity represents the set of all of its *ground instances*, which are equations derived from substituting terms for the variables. For instance, if s and t are terms, $s + t = t + s$ is an instance of the previous identity.

We will be concerned with a set of terms generated by some *constants*, $a, b, c, \ldots$, and some *function symbols*, $f, g, h, \ldots$ such that each function symbol has an arity (number of arguments) and, for instance, we can construct the term $f(s, t)$ if s and t are terms and f has arity 2.

A typical equational theorem proving problem looks like $I, E \vdash s = t$, where I is a set of identities and E is a set of equations, and we need a procedure to determine if the equation $s = t$ follows (indicated by the symbol $\vdash$) from I and E.

The distinction between equations and identities is important. Equations are easy, in the sense that the congruence-closure algorithm 1.2.2 is already enough to solve the problem entirely when we have only equations (and so $I = \{\}$).

Identities are what makes things tricky: given an identity $\forall x_1 \dots x_n.s = t$, we need to decide which substitutions for $x_1 \dots x_n$ we want to make in order to apply the identity. Given that there are infinitely many substitutions, deciding which ones to use is difficult, and in general, equational theorem proving with identities is undecidable (FiXme: citation needed?).

Still, using equality saturation (2.3), E-graphs provide a framework to tackle the problem: depending on the exact identities used, E-graphs can be used as a decision procedure, a semi-decision procedure, or an incomplete procedure for exploring the space of equalities.

E-graphs have a convenient feature: we don't need to provide the goal $s = t$ up-front! E-graphs will in general take the identities, equations, and possibly some terms of interest, and attempt to find all equations between relevant terms that hold. If equality saturation terminates, all possible equalities between the relevant terms has been discovered.

1.2.2 E-graphs for congruence-closure

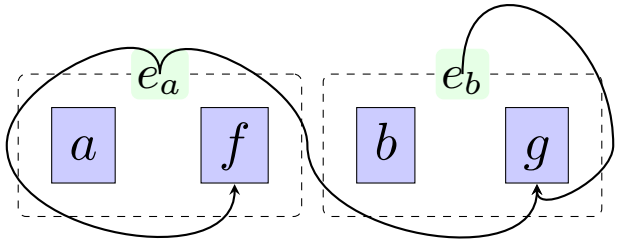
Consider the input set of equations

$$\begin{aligned} f(a) &= a, \\ g(f(f(a)), b) &= g(a, g(a, b)) \\ g(f(a), b) &= b \end{aligned}$$

The E-graph representing this set of equations is as follows:

Each dashed box is called an **E-class** contains multiple equivalent **E-nodes**. Each E-node doesn't represent just a term, but a set of terms: for instance, the E-class e_a contains the E-node $f(e_a)$, which represents the set of terms $\{f(t) \mid t \in e_a\}$. Indeed, this is an infinite set, containing $f(a), f(f(a)), f(f(f(a)))$, and so on.

The algorithm to build the E-graph to represent a set of input equations is known as **congruence-closure** and will be described in detail in section 2.2. Note that

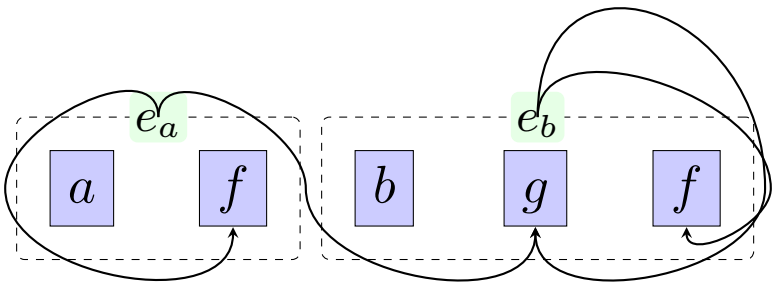


by 'following the arrows', the term $g(a, g(f(a), g(f(a), b)))$ is represented by the class e_b: this is not part of the input equations, but a consequence of them that can be read off from the E-graph once the congruence-closure algorithm has been applied.

1.2.3 E-graphs for equality saturation

Suppose instead of the equations $f(a) = a$ in the above example we had the identity $\forall x.\ f(x) = x$. To do equality saturation with this identity, we would *match* the left-hand-side $f(x)$ against our E-graph, which would find the matches $f(f(a))$ and $f(a)$, corresponding to the substitutions $x = f(a)$ and $x = a$ respectively. Then, we apply the same substitution to the right hand side, thus finding the *equations* $f(f(a)) = f(a)$ and $f(a) = a$. Inserting these equations into our E-graph and using congruence closure would then yield the above E-graph. Because equalities are symmetric, we would also have to find every match t for x and add the equation $f(t) = t$.

Equality saturation would terminate with the E-graph 1.2.3, as for every match in the E-graph of either side of $\forall x.\ f(x) = x$, the other side is in the E-graph and in the same E-class.



1.3 E-graphs and the problems of AC

FiXme: Diagrams need redoing. Wordiness fixed by commenting out things, for now.... I'm still unsure if this is the right example to be using for this section

While equality saturation on E-graphs can work with any set of identities, operators that are associative and commutative lead to a worst-case doubly-exponential explosion in the size of the E-graph when it is saturated. The **associativity** identity for an operator f is $\forall xyz.\ f(f(x,y),z) = f(x,f(y,z))$, and the **commutativity** identity is $\forall xy.\ f(x,y) = f(y,x)$.

Even if we do not run to saturation, it is likely that a large number of 'useless' associativity/commutativity steps will have been taken.

We will show some examples of E-graphs handling an AC function symbol $+$.

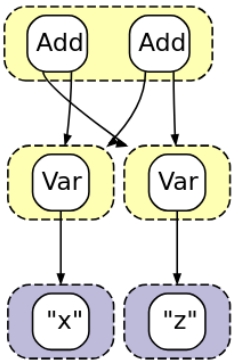


Figure 1: E-graph of the term $a + c$ after rewriting to saturation.

This graph shows that under AC we typically have both a lot of E-classes and a lot of E-nodes per class.

To give an example of a theory with both AC and 'interesting' equalities, we extend the theory from above (containing only the function Add and the AC equations) with a function f, satisfying $\forall xy.f(x+y)+1 = f(x)+f(y)$. We also allow integer constants, and allow the E-graph to reason about integer arithmetic, e.g. that $1+1=2$. We begin with the term $f(a)+(f(b)+f(c))$. We would like to discover that this input term is equal to $2+f(a+(b+c))$, using the reasoning

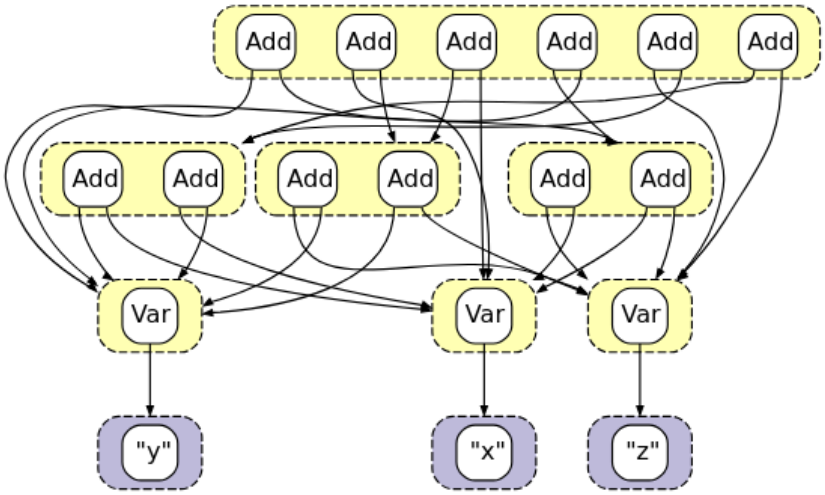


Figure 2: E-graph of the term $a + (b + c)$ after rewriting to saturation.

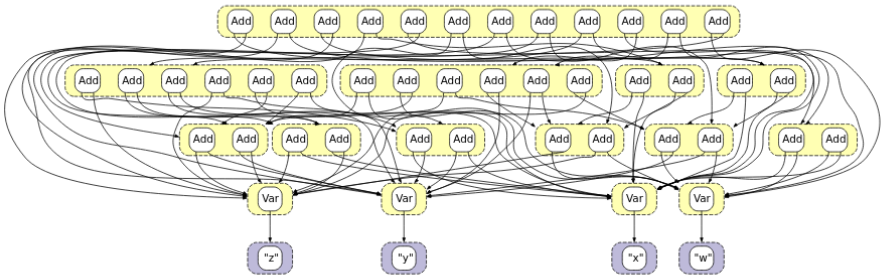


Figure 3: E-graph of the term $a + ((b + c) + d)$ after a number of rewriting steps.

$$
\begin{aligned}
&f(a) + (f(b) + f(c)) = \\
&f(a) + (f(b + c) + 1) = \\
&(f(a) + f(b + c)) + 1 = \\
&(f(a + (b + c)) + 1) + 1 = \\
&f(a + (b + c)) + (1 + 1) = \\
&f(a + (b + c)) + 2 = \\
&2 + f(a + (b + c))
\end{aligned}
$$

As we can see in Figure 4, the E-graph is on the way to discovering this, having for instance generated an equivalence class for the constant 2, but hits the demo's limit again before completing this proof. While this example may be a toy example on a rather limited demo, it shows the extent to which A/C can clutter up the E-graph with many nodes that are not essential to a desired proof. Our equational proof above only needs to use associativity twice and commutativity once.

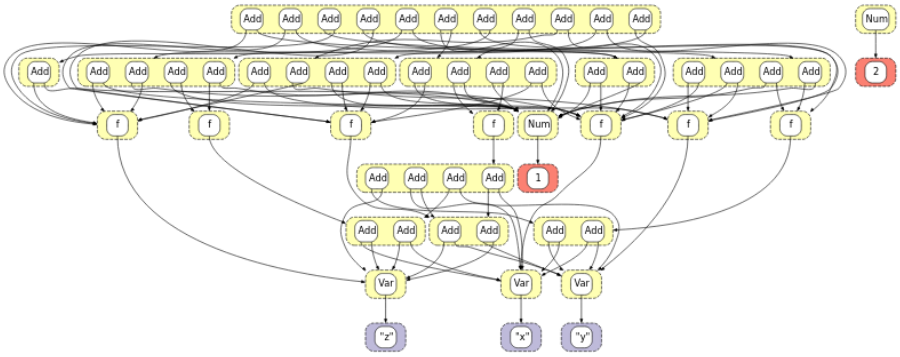


Figure 4: E-graph of the term $f(a) + (f(b) + f(c))$ after a number of rewriting steps.

1.4 Handling AC with EMTs: a first example

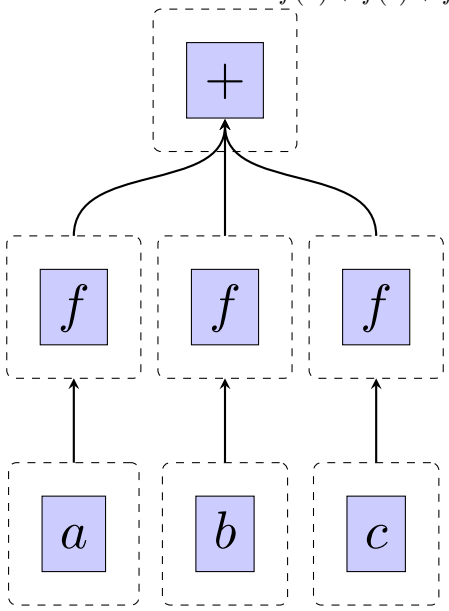
We will present a pictorial overview of how we might solve the above problem using EMTs rather than E-graphs.

The natural first step is to simply not add any different representations of terms that are equal up to AC; thus, nodes in the E-graph for AC operators will be variadic and we will consider their children unordered. This is the key idea from prior work ([9], [3], [4]).

Figure 5 begins with the variadic AC-term $f(a) + f(b) + f(c)$. We omit the labels of equivalence classes.

Now, in order to *match* the left-hand side of $f(x) + f(y) = f(x + y) + 1$, we need to do *matching modulo theories*, which will tell us of six matches: $(x, y) \in \{(a, b), (a, c), (b, c), (b, a), (c, a), (c, b)\}$. Then we will add the six substituted forms of $f(x + y) = 1$. Here, we will want to *insert modulo theories*, in a way that keeps the E-graph from representing multiple AC-equivalent terms. The arrows in Figure 6 have become crowded, but top E-class now represents the original term, and $f(a+b)+f(c)+1$ and its variants with a, b, c permuted.

Figure 5: EMT of the term $f(a) + f(b) + f(c)$.



Now, we once again try to match $f(x) + f(y)$. The only new matches will be $(x, y) = (a + b, c)$ and its permutations, and, modulo AC, the substituted-for term $f(x + y) + 1$ will be the same for all of these. So, we just need to add a single node.

Our last step is to notice that the term $f(a + b + c) + 1 + 1$ has, up to AC, the implicit subterm $1 + 1$, which we can reduce to 2. Having reduced the implicit subterm, we no longer need the 3-part term $f(a+b+c)+1+1$, and can entirely replace it with $f(a + b + c) + 2$.

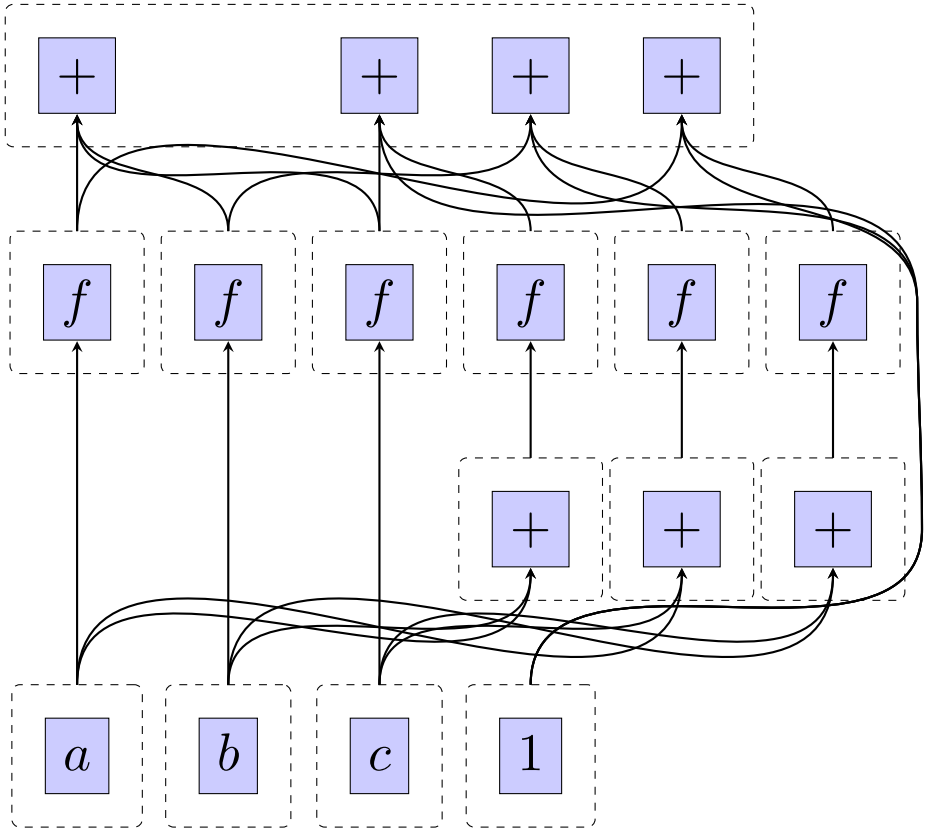
The final EMT (Figure 7) is saturated, and has 'discovered' our desired equality.

1.5 Challenges for EMTs

We have seen the promising aspects of EMTs, but there is a significant flaw in the description of EMTs shown so far, and we need something extra. Consider the equations

$$a + b = b \tag{1.1}$$
$$a + a = c \tag{1.2}$$
$$c + b = d \tag{1.3}$$

Figure 6: EMT of the term $f(a) + f(b) + f(c)$ after some steps.



taken modulo associativity of $+$.

We can represent them with an E-graph as follows:

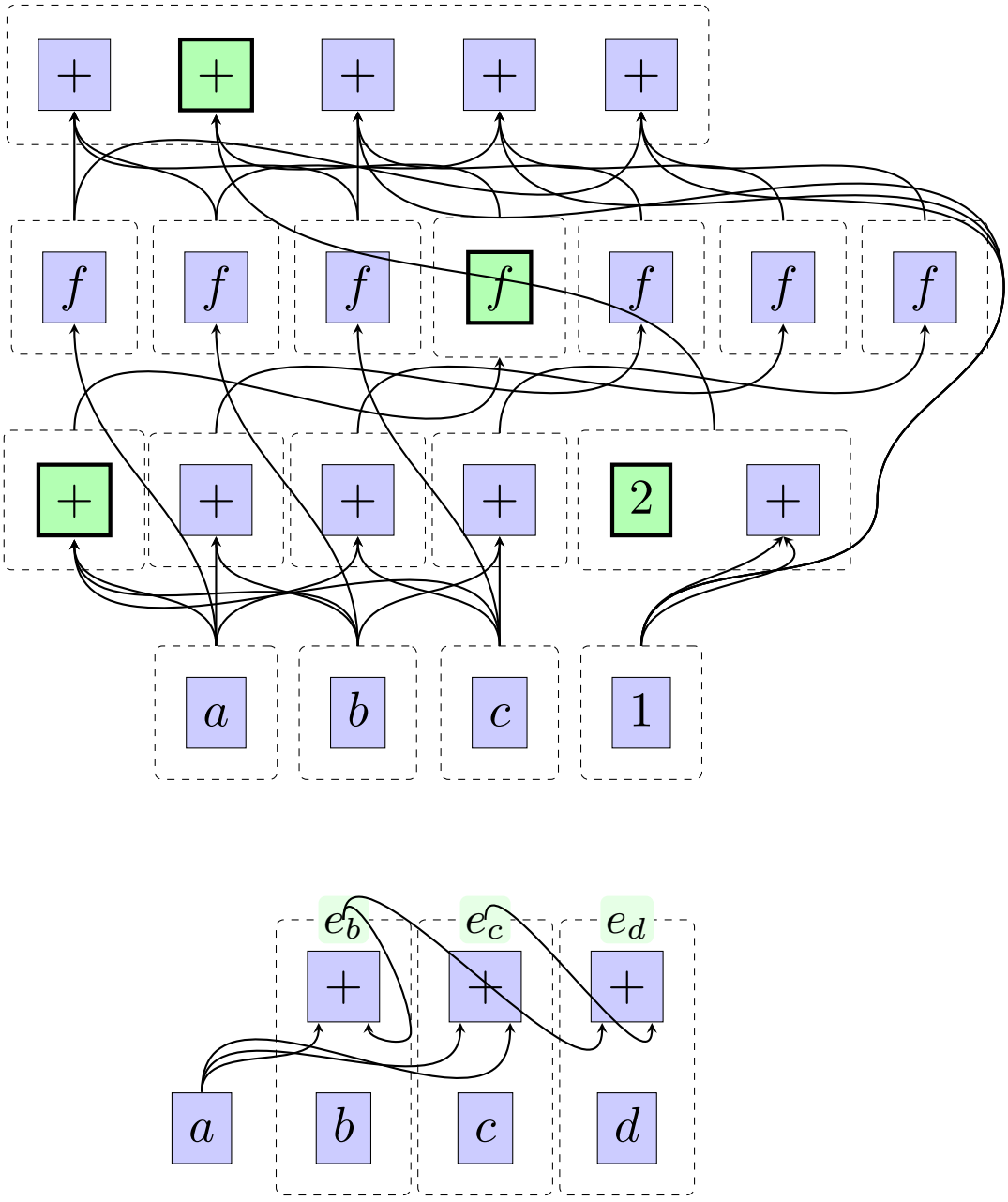
Consider the equation $b = d$, which follows from the simple deduction

$$b = a + b = a + (a + b) = (a + a) + b = c + b = d$$

However, e_b and e_d are two distinct E-classes in the E-graph! Why not?

The central step in the deduction relies on two different associations of the term $a + a + b$. The association $(a + a) + b$ is correctly assigned to class e_d, while the association $a + (a + b)$ is correctly assigned to the class e_b. However, given that we want to work modulo theories, we need to discover the fact that $e_b = e_d$, or we are throwing out some of the consequences derivable from the theory.

Figure 7: EMT of the term $f(a) + f(b) + f(c)$ once saturated, highlighting the desired term.



1.5.1 How E-graphs handle ordinary critical pairs, but not AC-critical pairs

To get a better perspective on this problem, we use ideas from rewriting and in particular the notion of Abstract Congruence Closure ([1]). We can think of E-graphs as presenting a rewriting system, and the above E-graph as containing the rewrite rules

$$\begin{aligned} b &\to e_b \\ c &\to e_c \\ d &\to e_d \\ a + e_b &\to e_b \\ a + a &\to e_c \\ e_c + e_b &\to e_d \end{aligned}$$

(A distinguished equivalence class for a is omitted to make the diagram easier to follow)

A critical pair for a rewriting system is a term that can be rewritten two different ways[1]. The perspective of Abstract Congruence Closure is that congruence closure, the core algorithm for 'rebuilding' E-graphs, constructs a *confluent term rewriting system* on an extended signature. Handling critical pairs is required for confluence, which ensures that a term rewrites into a single result; if we have a term t that rewrites to both e_a and e_b, we ought to automatically *deduce* that the E-classes e_a and e_b should be merged.

By introducing new constants $e_1, e_2, \ldots$ as E-ids, we can 'flatten' an input term $f(\ldots, g(h(\ldots)), \ldots)$ into just $f(\ldots, e_i, \ldots)$ and then assign it the name e_k for some k.

Now, to ensure the convergence of the rewrite system, we can look at two kinds of cases where a term could rewrite in two ways:

$e_1 \leftarrow f(\ldots e_i \ldots) \to e_2$ in which case, we ought to deduce that $e_1 = e_2$. This corresponds to merging E-classes which would contain duplicates of the same E-node.

$f(\ldots e_i \ldots) \to e_1$ and $e_i \to e_2$, in which case we ought to deduce that $f(\ldots e_2 \ldots) = e_1$. This corresponds to canonizing the E-nodes using the union-find structure.

By fixpoint propagation, resolving these two cases are enough to resolve all cases of critical pairs with overlaps on more deeply nested subterms.

[1] It is also required to be minimal among such terms, but we gloss over this here

E-graph rebuilding can handle ordinary critical pairs, but this doesn't cover our above example, which is an *AC-critical pair*.

1.5.2 Equality satuation: handling AC-critical pairs by brute force

E-graphs (not modulo theories) handle AC-critical pairs the same way as critical pairs for any other theory, with equality saturation: we *ground* the AC identities by matching either side with the E-graph, and adding the equivalent other side (if it is not present) and merging the equivalence classes corresponding to either side.

Why do E-graphs fare poorly with AC? The two identities, (A) $\forall xyz.\ x + (y + z) = (x + y) + z$ and (C) $\forall xy.\ x + y = y + x$, can be semantically characterized as saying that two AC-terms are equal if they are equal as non-empty multisets of their subterms. Equality saturation will handle these identities by eventually deriving that an input term $a_1 + a_2 + \ldots + a_n$, thought of as the multiset $\{a_1, a_2, \ldots, a_n\}$, is in the same E-class as every E-node of the form $s + t$ where s and t partition the multiset $\{a_1, a_2, \ldots, a_n\}$. There are $2^n - 1$ non-empty multisets s and t which we can partition a starting multiset of size n into[2], and one E-class will need to be generated for each. So, to reach saturation, we must create $O(2^n)$ E-classes, and it is easy to see that each E-class will, on average, have its own $O(2^n)$ E-nodes to represent every multiset partition of the multiset it represents.

These 'redundant' E-nodes and E-classes play a role: any of the $O(2^{2^n})$ AC-subterms of the input could be part of an AC-critical pair, and so equality saturation eagerly assigns new E-classes to them, just in case. Introducing these extra E-classes and E-nodes is seemingly the best we can do in general unless we have a clever way to detect which ground instances of the identities are needed in order to ensure confluence modulo those identities.

In our above example, the associative identity would match against $a + (a + b)$, which is a term represented in our E-graph, and then merge this term (assigned to equivalence class e_b) with the corresponding other side of the associativity identity (assigned to equivalence class e_d *if* we have also added the instance of commutativity $c + b = b + c$ to the graph. Otherwise, adding this instance of commutativity later on and then rebuilding will correctly find the need to merge these classes).

1.5.3 EMTs: Handling AC-critical pairs less naïvely

We don't need to add all AC-subterms to find critical pairs. It suffices to consider every pair of E-nodes with an AC symbol and check them for 'overlap'.

[2]If the multiset has an element of multiplicity more than 1, some of these will be the same, so this number will be reduced. However, we consider the worst case

In our example, the E-nodes $a + e_b$ and $a + a$ form a potential critical pair $a + a + e_b$ as they overlap on the subterm a. We see that this implies the equation $a + e_b = e_c + e_b$, which can be added to the E-graph, which will merge the classes e_b of the LHS and e_d of the RHS.

This procedure always terminates ([1]). It is, in fact, a special case of Buchberger's algorithm for finding Gröbner bases of sets of polynomials ([7]), a well-studied algorithm with a lot of work done in optimizing it to be fast enough in practice.

Chapter 2

E-graphs in depth

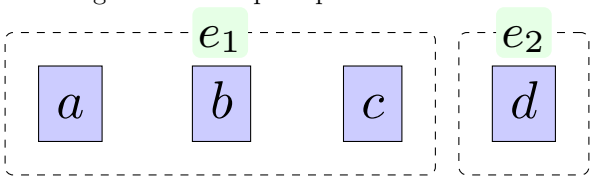
FiXme: This needs a pass to improve the quality. For now, it is something of a dumping ground of 'elementary' material

2.1 Equivalence relations and Union-find

The handling of E-classes, which are equivalence relations of E-nodes, is a central part of the E-graph data structure.

Figure 1 shows a union-find structure that assigns the name e_1 to the equivalence class that encompasses the terms a, b, and c, and the name e_2 to the equivalence class containing just the term d.

Figure 1: A simple equivalence relation



If we abstract from the contents of the E-nodes, we can just think of each E-node as a unique *constant*. Then the problem is to determine if an equation between constants $a = b$ follows from a set of equations $\Gamma = \{c = d, \ldots e = f\}$. For this problem, we have a beautiful solution that comes with an efficient implementation. This is what we would love to see for every fragment, but things become more difficult as we introduce more complexity.

We recall some standard notions. Given a set, an **equivalence relation** over

that set is a reflexive, symmetric, and transitive relation. Given an equivalence relation, we can take the **quotient** of the set by it, a set containing a single element for each **equivalence class** of elements in the original set.

The *Union-Find* data structure is an efficient means to represent an equivalence relation over a set, and to update it either by extending the underlying set or by merging two equivalence classes. Union-find avoids any fiddly manipulation with reflexivity, symmetry, and transitivity, and focuses on a *semantic* characterization of equivalence relations: a function $r : A \to B$ splits the domain A into equivalence classes based on the equality of their images in B: define R by $R = \{(a, b) \mid r(a) = r(b)\}$. In this, B *is* just the set of equivalence classes, and r *is* just the quotient.

Using this fact, we can sketch an implementation that maintains this mapping r from a set of constants to a set of equivalence classes. To start off with, we don't need to represent the codomain B in any fancy way: we just assign a new identifier to each equivalence class. Then, a basic implementation is just an array or table with indexes in A and entries in some set of identifiers for equivalence classes.

This representation potentially requires a bit of work to update: if we want to take into account a new equation $a = b$ with an existing r, we need to set all things mapping to $r(a)$ and $r(b)$ to the same identifier. It is possible to lazily update the union-find whenever doing a find, effectively storing a function q whose fixed-point is r, which makes the running time for n union and find operations be just $O(n\alpha(n))$, where α is the inverse Ackermann function, a very slow-growing function FiXme: source.

2.2 Congruence relations and Congruence-closure

A congruence relation on a set of terms is one that respects the function symbols: for instance, if f is a function symbol of arity 3, and $s_1 = t_1$, $s_2 = t_2$, and $s_3 = t_3$, then $f(s_1, s_2, s_3) = f(t_1, t_2, t_3)$. In general, we get the following axiom schema: for each n-ary symbol f, $a_1 = b_1, \ldots, a_n = b_n \vdash f(a_1, \ldots, a_n) = f(b_1, \ldots, b_n)$. This is the larger fragment solved by a congruence-closure data structure.

We keep a union-find storing equivalence classes of **E-nodes**[1]: as we have seen, if $s = t$, then we don't need to waste space represent both $f(s)$ and $f(t)$, given that these are identical. So, we restrict ourselves to keeping not a set of terms, but a set of 'lifted' terms like $f(e_1)$, if e_1 is the name we assign to the class containing s and t.

This has the potential to save an infinite amount of space: if we have the input equation $f(a) = a$, then the equivalence class of a up to congruence includes

[1] 'E' in E-graph, E-node, E-class, etc., stands for equality

the infinite set of terms $a, f(a), f(f(a)), \ldots$. So the 'lifting' of terms to act on equivalence classes lets us represent every congruence relation in a finite way.

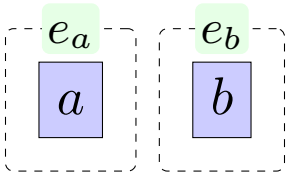
Indeed, this idea yields the congruence-closure algorithm for deciding equality over any set of equations. We will simply refer to the congruence-closure structure as an **E-graph**, although the term E-graph is commonly used when we also include equality saturation (2.3).

We will show the congruence-closure algorithm running on our previous set of equations

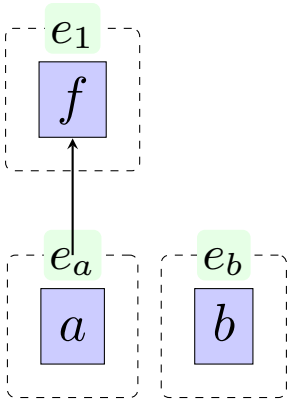
$$\begin{aligned} f(a) &= a, \\ g(f(f(a)), b) &= g(a, g(a, b)) \\ g(f(a), b) &= b \end{aligned}$$

We can build the E-graph over these equations as follows:

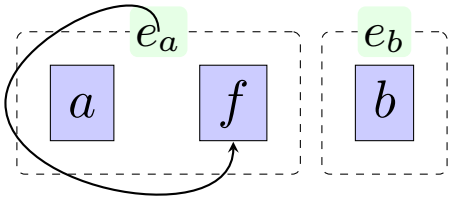
First, we add all terms with no subterms, namely a and b. These each get a distinct equivalence class.



Next we add $f(a)$: a is represented by e_a, so we represent $f(a)$ by $f(e_a)$, and can graphically depict this with an arrow:

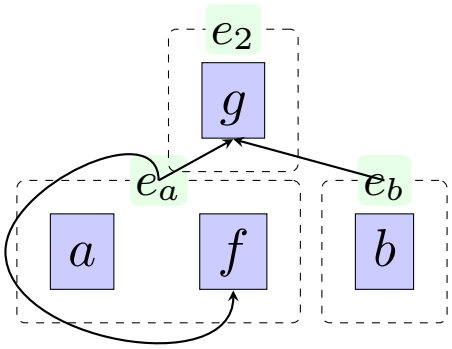


Now we have represented both terms in the equation $f(a) = a$, so we merge the resulting equivalence classes:



This merged class represents all of $a, f(a), f(f(a)), \ldots$.

Now we look at representing some of the input terms using the function symbol g. Using our existing equivalence classes, $f(f(a))$ is represented by $f(f(e_a)$ is represented by $f(e_a)$ is represented by e_a, so $g(f(f(a)), b)$ is represented by $g(e_a, e_b)$, which is not yet in our E-graph. $g(e_a, e_b)$ represents all of $g(f(f(a)), b)$, $g(a, b)$, and $g(f(a), b)$.



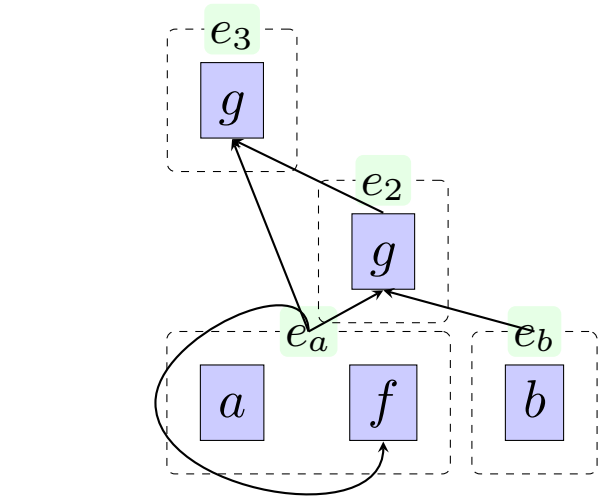
The last term to represent is then $g(a, g(a, b))$, which is represented by $g(e_a, e_2)$.

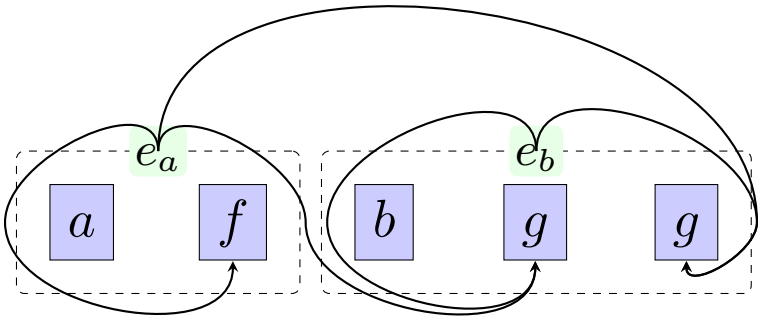
Now, we need to process the remaining two equations. $g(f(f(a)), b) = g(a, g(a, b))$ tells us to merge the equivalence classes e_2 and e_3. $g(f(a), b) = b$ tells us to merge e_2 and e_b.

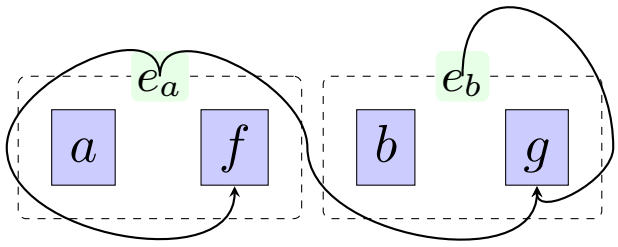
The last step is to apply to rule of *congruence*. Our graph now has two nodes representing $g(e_a, e_b)$. We merge them; as they are already in the same class, there is no further work to be done:

However, in some cases applying congruence can cause a chain of merges. This process terminates as there is a maximal state of finite size (an E-graph assigning all nodes to the same equivalence class), so any potential infinite chain of congruence steps will reach a fixed point after finitely many steps.

Note the final E-graph has two *cycles*: $f(e_a) = e_a$ and $g(e_a, e_b) = e_b$. In general,


e_3
g
e_2
g
e_a
e_b
a
f
b


e_a
e_b
a
f
b
g
g


e_a
e_b
a
f
b
g

we might have cycles going through multiple intermediary nodes: $f(\ldots g(\ldots e_1 \ldots) \ldots) = e_1$. Cycles become very important in the analysis of how theories fare with E-graphs under equality saturation (section 3).

For congruence closure, and thus E-graphs, the laziness possible in the union-find seems difficult to make use of, as we need to detect the congruence $f(a_1, \ldots, b, \ldots, a_n) = f(a_1, \ldots, b, \ldots, a_n)$ once b and c have been equated; it is not enough to just 'discover' that $b = c$ when calling find on b or c or one of their equivalents, as such a find call is not guaranteed to happen at any particular point before we need to discover the congruence.

2.2.1 Congruence closure builds a rewrite system

We can read off a set of equations from our final E-graph:

$$\begin{aligned} a &= e_a \\ f(e_a) &= e_a \\ b &= e_b \\ g(e_a, e_b) &= e_b \end{aligned}$$

Given this resulting set of equations, we can determine if any two terms at all are equal under our initial set of equations: simply reduce the terms by treating these equations as a *rewrite system*, read from left-to-right, and see if the normal forms are the same. FiXme: Refs for connections to rewrite systems.

2.3 Equality saturation

Now that we can reason about the logical consequences of (ground) equations, we look to solve the problem of reasoning with identities. The method of **equality saturation** is what we will focus on in this thesis.

The key idea is that, given a representation of a congruence relation, we *E-match* each identity $\forall a_1, \ldots, a_n. s_1 = s_2$ against the congruence relation to find all instances $s_1[t_1/a_1, \ldots, t_n/a_n]$ and $s_2[t_1/a_1, \ldots, t_n/a_n]$. Then, we find the representatives of these two terms. If they are not equal, we have found two equivalence classes to merge to take into account the identity. We will refer to this as a **proper merge**.

Unfortunately, proper merges are not sufficient. If we consider the associative identity $\forall abc.\ a + (b + c) = (a + b) + c$, and we begin with the two equalities $u = w + (x + (y + z))$ and $v = ((w + x) + y) + z$, we would like to prove the theorem that

$$
\begin{aligned}
(\forall abc.\ a + (b + c) = (a + b) + c),\\
u = w + (x + (y + z)),\\
v = ((w + x) + y) + z\\
\vdash u = v
\end{aligned}
$$

However, the attempt to prove even the first step, that $w + (x + (y + z)) = (w + x) + (y + z)$, founders. Our representation includes no equivalence class to represent $(w + x) + (y + z)$, and so there are no equivalence classes to merge.

The 'solution' is to introduce the possibility of **inserting** new classes during the procedure. For instance, we can find an instance $s_1 = s_2$ of an identity such that s_1 maps to a known equivalence class but s_2 doesn't. We seed s_2 as a new term by calculating its equivalence class, potentially introducing a new class if no prior one exists, and then merging the equivalence classes of s_1 and s_2. We call this mix of seeing and a proper merge an **improper merge**.

Insertion has one key problem: we now have no guarantees the process of merging will terminate. With only proper merges, there are a finite set of input terms and thus a finite bound on the equivalence relation we maintain. Thus, eventually we will get to a point where no merge changes the equivalence relation, and we have reached **saturation**. Introducing new classes for inserted terms now means that saturation is no longer guaranteed to be reached at any point, and worse, the strategy for insertion and merging now can affect whether or not saturation is reached.

To even get a *semi-decision procedure*, where we aim to prove a specific equality if it exists and can fail otherwise, we must ensure that the insertion and merging processes are sufficiently *fair* (in the sense of 'fair' as used in concurrent programming): we must have some strategy to ensure every possible term that could be inserted *does* get inserted eventually, or we might get stuck inserting other terms infinitely while nonetheless not making progress on the goal.

In some sense, this is to be expected, as even for just equational theorem proving, there are theories which are known to be undecidable. However, even when the process of E-matching, inserting and merging doesn't terminate, we can consider the infinite fixed-point of this process as the 'semantics' of our E-graph [6], and this will be what we will preserve in the shift to EMTs. This process will be referred to as the 'equality saturation outer loop'. A schematic form is as follows:

1. Insert all the starting terms into the E-graph
2. Repeat until fixed point:
   (a) **E-match** an identity against the E-graph, finding all ground in-

stances of one side of the identity that exist in the E-graph.

(b) **Insert** Using the substitution from the previous step, construct the term resulting from applying the substitution to the other side of the identity. If it is not already represented by the E-graph, add it to the E-graph.

(c) **Merge** the E-classes representing both sides of the equality above.

(d) **Rebuild** the graph, using the rules of congruence closure

If the fixed point is reached, we call the graph **saturated**.

2.3.1 A few variations

We can do several matches/insertions/merges in between rebuilding; it is sufficient that rebuilding is *eventually* run after changes to the E-graph [8].

There are plenty of extensions to the outer loop that include things such as conditional reasoning (along the lines of: if $a = b$, then assert that $c = d$ by merging classes) or E-class analyses ([8]).

Chapter 3

Which theories do E-graphs struggle with?

AC is a good example of a theory which E-graphs handle poorly. However, this is not necessarily the case for every common theory, so it may not always be worth the extra complexity of using EMTs. We list some common identities which are worth analyzing:

Theory name	Abbreviation	(Function symbol, arity)	Theory
Associativity	A	$(f, 2)$	$\forall xyz.f(x, f(y, z)) = f(f(x, y), z)$
Commutativity	C	$(f, 2)$	$\forall xy.f(x, y) = f(y, z)$
Left-unitality		$(f, 2), (e, 0)$	$\forall x.f(e, x) = x$
Right-unitality		$(f, 2), (e, 0)$	$\forall x.f(x, e) = x$
Unitality	U	$(f, 2), (e, 0)$	Both of the above
Left-distributivity		$(f, 2), (g, 2)$	$\forall xyz.g(x, f(y, z)) = f(g(x, y), g(x, z))$
Right-distributivity		$(f, 2), (g, 2)$	$\forall xyz.g(f(y, z), x) = f(g(y, x), g(z, x))$
Distributivity	D	$(f, 2), (g, 2)$	Both of the above
Idempotence	I	$(f, 2)$	$\forall x.f(x, x) = x$
Inverse	N	$(f, 2), (i, 1), (e, 0)$	Unitality and $\forall x.f(i(x), x) = e$ $\forall x.f(x, i(x)) = e$
Involution	V	$(i, 1)$	$\forall x.i(i(x)) = x$

Table 3.1: Definitions of some common identities

We use the convention of naming combined theories by taking the juxtaposition of the theory names: AC for a function symbol f is the theory with both A and C identities, and so forth.

Chapter 4

E-graphs modulo theories

To deal with problematic theories such as AC, we will define the notion of *E-graph modulo theory*. As we have seen, E-graphs deal with AC by explicitly generating every term that is AC-equivalent to some input term, which is always a finite but worst-case doubly-exponential-sized set.

To build up to the notion of EMTs, we consider *critical pairs*, a notion from term rewriting. If we have a rewriting relation $\rightarrow_{\mathcal{R}}$, then a **peak** is a trio of terms s_1, s_2 and t such that $s_1 \leftarrow_{\mathcal{R}} t \rightarrow_{\mathcal{R}} s_2$. A **critical pair** is a peak that cannot be obtained as a non-trivial substitution instance of another.

For instance, if we have the rules $f(x, y) \rightarrow_{\mathcal{R}} g(y, y, x)$ and $h(x) \rightarrow_{\mathcal{R}} x$, then the term $f(h(g(x, y, z)), w)$ is a peak, because

$$g(w, w, h(g(x, y, z))) \leftarrow_{\mathcal{R}} f(h(g(x, y, z)), w) \rightarrow_{\mathcal{R}} f(g(x, y, z), w)$$

Now, E-graphs take a set of equations E and build a rewrite system $\rightarrow_E$ such that $s =_E t$ iff there is a c such that $s \rightarrow_E^* c \leftarrow_E^* t$. If we have a rewriting system with a peak $s_1 \leftarrow_{\mathcal{R}} t \rightarrow_{\mathcal{R}} s_2$ and we want to make $\mathcal{R}$ be able to describe a theory of equality, we need to find some way to enforce **confluence**: because s_1 and s_2 ought to be equal if $\mathcal{R}$ is taken to be axiomatizing equalities, we need to ensure there is a term c such that $s_1 \rightarrow_{\mathcal{R}}^* c \leftarrow_{\mathcal{R}}^* s_2$. Ground Knuth-Bendix completion FiXme: cite is one method to do this, based on adding a new rewrite $s_1 \rightarrow_{\mathcal{R}} s_2$ to 'orient' each critical pair.

E-graphs take another approach: a nonconfluent critical pair $s_1 \leftarrow t \rightarrow s_2$ can only arise if there are rewrite rules that *overlap*, which we can put into two categories: if $f(\ldots) \rightarrow s_1$ and $f(\ldots) \rightarrow s_2$ can both apply to the same ground term, this will manifest in an E-graph as an E-node that is part of two E-

classes, which will be merged on rebuilding. If $f(\ldots, g, \ldots) \to s_1$ and $g \to s_2$, then note that we will not actually represent the term $f(\ldots, g, \ldots)$ directly: rather, we will introduce a new name e_1 and write it as $f(\ldots, e_1, \ldots) \to s_1$. In this case, after taking into account the rewrite applied to g, the critical pair is resolved automatically, as if $g = s_2 = e_1$ then $f(\ldots, g, \ldots) = f(\ldots, s_2, \ldots) = f(\ldots, e_1, \ldots) = s_1$ will all follow from the presented equational theory.

So, the flattening that happens when building E-graphs can be seen as a process of introducing new names for every subterm, since every subterm is a *potential* critical pair. By introducing these names, we restrict the need to scan for critical pairs to the case of complete overlap: two equal terms in different E-classes.

Based on this, we can diagnose the problems of E-graphs on AC as the E-graph speculatively adding all 'AC-subterms' to the E-graph as any one of them could part of some critical pair, up to AC. For instance, if we have the terms $a+b+c+d$ and $c+e+b$, they have an overlap in the AC-subterm $b+c$, and if these terms were assigned to different E-classes, we would have the AC-critical pair $e_1 + e \leftarrow a+b+c+d+e \to a+d+e_2$.

So, while E-graphs are built around presenting a confluent ground rewriting system, the notion of EMT over a theory T suggests moving to the notion of confluent ground T-rewriting system in the natural way, in which case we need to pay special attention to critical pairs, which need to be resolved to maintain confluence.

4.1 EMTs: prior attempts

That *some* notion of EMT ought to exist has been stated multiple times before ([9], [3], [4]). The hope is to be able to use theory-specific knowledge to integrate certain identities directly into the procedures, rather than having to time-consumingly match, seed, and merge as directed by the identities. We will briefly go over the key intutition behind these proposals, and show that they fail to take into account the T-critical pair idea we outlined above.

4.1.1 Intuition 1: Canonizers

Zucker [9] notes that we have *canonizers* for many theories, which provide a way to solve the problem of ground equality modulo theories. A canonizer for a theory T is just some function on terms c such that, if $t_1 =_T t_2$, then $c(t_1) = c(t_2)$: that is, it shifts the problem from equality modulo T to just ordinary term equality.

4.1.2 Intutition 2: Containers

The signature for an E-graph, Σ, is equivalent to a certain kind of *container*: one that has a number of 'shapes' (corresponding to the function symbols) and

each shape has an ordered collection of slots (with length given by the arity of the shape). Thus, it is natural to consider generalizing this to containers of more general varieties. If one assigns to a single symbol one shape of every arity , with ordered slots, this gives the theory of AU: terms like $+()$, $+(a, b)$, $+(a, b, c)$ are all covered by this notion, and one doesn't need to arbitarily split a given list into binary uses of $+(-,-)$ and nilary uses of e. If the ordering constraint is dropped on the collections of slots, we get multiset containers, suitable for ACU, and moving to set containers gives something akin to ACIU – ACU with idempotence.

The second intuition also arises naturally out of concrete implementation work on E-graphs. Given a signature with an ACU binary operator f with unit e, and some other operators g, h, we can imagine building a parameterized data structure for term-fragments along the lines of the following pseudo-Haskell:

```
data TermF a
       = E
       | F a a
       | G a a a
       | H a
```

Looking at this structure and considering the ACU nature of f and e, it is only natural to change this to

```
data TermF a
       = Fs (Multiset a)
       | G a a a
       | H a
```

for an appropriate type constructor of multisets.

FiXme: Cite literature on polynomial functors/containers?

4.1.3 Canonizers and containers and the problem of flattening

The first intuition naturally connects to the second intuition if we consider the generalized idea of 'container' as representing some canonical form. Both approaches at first glance seem to take care of a lot of potential fiddling with identities in such theories. However, there is one issue: while we manage to remove the some part of the theory with this approach, there is still one derived identity which we need to handle.

For instance, consider an AU pair of operators $+, 0$. The identities are

$$a + (b + c) = (a + b) + c \tag{4.1}$$
$$a + 0 = a \tag{4.2}$$
$$0 + a = a \tag{4.3}$$

Using a list representation, these become

$$[a, [b, c]] = [[a, b], c] \tag{4.4}$$
$$[a, []] = a \tag{4.5}$$
$$[[], a] = a \tag{4.6}$$

These identities are not entirely automatic just by switching to lists: we need the additional embedding and and flattening laws

$$a = [a] \tag{4.7}$$
$$[[a_1, \ldots, a_n], [b_1, \ldots, b_m], \ldots] = [a_1, \ldots, a_n, b_1, \ldots, b_m, \ldots] \tag{4.8}$$

Or in other words, every term can be embedded in a list of just that term, and a list of lists can be flattened by concatenating all of its elements.

Eq 4.7 and eq 4.8 together with the basic rules of lists are what is required to prove eq 4.4, eq 4.5, and eq 4.6.

FiXme: Cite literature on unbiased presentations? This is a monad law...

It is the need to handle embedding and flattening that gives rise to problems with an approach based on just canonizers or containers. We will then also see that embedding and flattening are the two rules that govern the formation of T-critical pairs.

A typical example of where these approaches break down is given by

$$a + b = b \tag{4.9}$$
$$a + a = c \tag{4.10}$$
$$c + b = d \tag{4.11}$$

taken modulo associativity of $+$.

The goal is to prove simply that $b = d$, which follows from the simple deduction

$$\begin{aligned} & b \\ &= a + b \\ &= a + (a + b) \\ &= (a + a) + b \\ &= c + b \\ &= d \end{aligned}$$

If we try the containers approach, we start by building up an E-graph with the following E-classes:

$e_1 = \{b, [a, b]\}, e_2 = \{c, [a, a]\}, e_3 = \{d, [c, b]\}$.

However, this is by itself, entirely inert. We need to use flattening to reason that, since $b = [a, b]$, that $[a, b] = [a, [a, b]] = [a, a, b]$. Then we need to use the converse of flattening to notice that $[a, a, b] = [[a, a], b] = [c, b] = d$.

For the canonizer approach, we can easily canonize the input equations: indeed, all of them have both sides in canonical form already. What is missing, then? Well, if we view the E-graph as a rewrite system, there is of course an A-critical pair between the rewrite $a+b \to e_1$ and $a+a \to e_2$. The critical pair is $a+a+b$, which can be rewritten as either $a + e_1$ or $e_2 + b$.

We conclude that canonizers and containers are not enough: they can rewrite terms that might appear in equations to a canonical form, but this is not enough to make the equations $s_i = t_i$ when flattened and oriented into the form $s_i \to^* e_j \leftarrow^* t_i$ to naturally form a T-confluent system rewrite.

There are some cases where canonizers or containers avoid this problem: for instance, consider the theory C. A critical pair mod C can only be of the form $e_1 \leftarrow a+b =_C b+a \to e_2$. If we always canonize (mod C) both sides of equations when we orient them (say, by sorting according to some stable order), then we will have a confluent system mod C. (This requires some care to maintain the ordering given that a and b might be E-class ids and subject to change via union-find and rebuilding).

FiXme: Backward ref to minimally expansive theories, to be renamed to weak acyclicity or whatever. Actually, conjecture: all minimally expansive/weakly acyclicity theories can be done with just a canonizer/container

4.2 EMTs and the word problem

4.2.1 The problem of matching modulo theories

Still, even with a canonizer or container, there is the problem that E-graph matching becomes matching modulo theories.

FiXme: Expand this. For theories where a canonizer works/solves the word problem, is Zucker's bottom-up idea sufficient?

4.3 E-graphs modulo theorie

4.3.1 Defining EMTs

Our general setup will be similar to E-graphs: we will maintain a set of equivalence classes, which will contain E-nodes/term fragments over equivalence classes.

For a theory T, the T-part of the E-graph is the set of E-nodes that use a function symbol that appears in an identity in T.

For instance, if we have the E-class $e_1 = \{e_2+e_3, f(e2), e_3+e_4\}$, then the T-part of this, taking T to include only the $+$ symbol, is $e_1 = \{e_2 + e_3, e_3 + e_4\}$. An E-class that has no T-nodes still will appear in the form of $e_2 = \{\}$, recording that e_2 is T-*atomic*.

Our definition of EMTs will require a *decision procedure for the T-word problem*. That is, we need a procedure that, given the union of T and a set of equations, determines if another arbitrary equation follows from T and the equations.

FiXme: Change to include weak EMTs for A etc

Necessity of the word problem 1

We might imagine that we could restrict to the problem of determining if $e_i = e_j$ for equivalence classes e_i, e_j that actually appear in the E-graph. However, this restriction does not make anything easier: given an arbitrary equation $s = t$, we can insert s and t into the E-graph and then ask if the resulting equivalence classes are equal.

Necessity of the word problem 2

An E-graph in canonical form decides the word problem over uninterpreted function symbols. More concretely, an E-graphs describes a canonizer c such that $c(s) = e_i = c(t)$ iff $s = t$ follows from the input equations. It is also the case that, say, $c(f(s)) = f(e_i) = c(f(t))$ can be deduced by the E-graph even if $f(e_i)$ is not represented, but we can weaken this requirement for EMTs. We

will consider just the very weak requirement that $s \to e_i \leftarrow t$ iff $s = t$ follows from the input equations and T, where $\to$ is a sequence of either rewrite rules from the EMT and applications of identities in T (and symmetrically for $\leftarrow$).

FiXme: Writing this makes me think we really ought to introduce $\vdash$ at some point for brevity. Also, check later if we have commented on the sufficiency of our requirements

Unfortunately, this rules out EM(A) and EM(AU), given that the relevant word problems are undecidable.

Also, we need to extend matching to the more general matching modulo T, given that the T-part now won't necessarily represent all terms up to T in a 'direct' way. (The hope of being able to represent E-graphs under T in less space than standard E-graphs is, of course, the motivation for EMTs to begin with.)

Chapter 5

E-graphs modulo AC

FiXme: Make sure we have cited 'congruence closure mod AC' by Bachmair and the Cochon paper as very relevant prior work. Tiwari also

5.1 Solving the word problem modulo AC

A single AC operator forms a *commutative semigroup*. We have a remarkably tight bound on the asymptotic complexity of solving the word problem over commutative semigroups: Mayr and Meyer [2] prove that it is EXPSPACE-complete, that is to say, it requires $O(2^n)$ space to compute. This result also provides an embeddding of the word problem into the *polynomial ideal* word problem, a problem that is solvable via finding *Gröbner bases*. Finding Gröbner bases is central problem in computer algebra which is also EXPSPACE-complete, and so it is both asymptotically optimal for solving AC and, we hope, the best-optimized approach *in practice* for solving the AC word problem. Moreover, the same algorithm can be used for solving various variants of AC, such as ACU, commutative rings, and polynomials over commutative rings. Therefore, we focus on Gröbner basis methods as a key method that provides a word-problem solver for a wide range of theories of interest.

5.1.1 Polynomial ideals and the AC embedding

Take a finite set $X = \{e_1, \ldots, e_n\}$. Then two terms over X built from an AC operator f are AC-equal if they have the same number of each element of X in any order; that is to say, we take them equal as *multisets*.

Instead of writing a term as $f(e_1, f(e_1, f(e_2))$, we will instead write it as $e_1^2 e_2$, to connect better with the standard notation for polynomials.

The set of **polynomials** $R[X]$ over X and the ring of coefficients R and is the set of R-linear combinations of AC terms over X: for instance, if $r_1, r_2 \in R$, then $r_1 e_1^2 e_2 + r_2 e_3$ is a polynomial in $R[X]$.

Given a finite set of equations E between X-terms of the form $\alpha_i = \beta_i$, we can construct the set $I(E) = \{\sum_i g_i(\alpha_i - \beta_i) \mid g_i \in R[X], E_i = (\alpha_i = \beta_i)\}$.

Then, E implies that two X-terms s and t are equal iff $s - t \in I(E)$.

FiXme: TODO: proof

$I(E)$ is an ideal in the set of polynomials

5.1.2 Gröbner bases for ideals

5.1.3 Computing Gröbner bases via Buchberger's algorithm

5.2 Matching modulo AC

Chapter 6

An implementation

Chapter 7

Evaluation

Chapter 8

Prior work

Chapter 9

Conclusion and future work

Bibliography

[1] Leo Bachmair and Ashish Tiwari. "Abstract Congruence Closure and Specializations". In: *Automated Deduction - CADE-17*. Ed. by David McAllester. Berlin, Heidelberg: Springer, 2000, pp. 64–78. ISBN: 978-3-540-45101-3. DOI: 10.1007/10721959_5.

[2] Ernst W Mayr and Albert R Meyer. "The complexity of the word problems for commutative semigroups and polynomial ideals". In: *Advances in Mathematics* 46.3 (Dec. 1, 1982), pp. 305–329. ISSN: 0001-8708. DOI: 10.1016/0001-8708(82)90048-2. URL: https://www.sciencedirect.com/science/article/pii/0001870882900482 (visited on 03/11/2026).

[3] Pavel Panchekha. *LIN-graphs, an Egraph modulo T*. Pavel's Blog. May 23, 2025. URL: https://pavpanchekha.com/blog/lin-graphs.html.

[4] Saul Shanabrook. *Custom Data Structures in E-Graphs*. PLSE Blog. Feb. 24, 2026. URL: https://uwplse.org/2026/02/24/egglog-containers.html.

[5] Natarajan Shankar. "Little Engines of Proof". In: *FME 2002:Formal Methods—Getting IT Right*. Ed. by Lars-Henrik Eriksson and Peter Alexander Lindsay. Red. by Gerhard Goos, Juris Hartmanis, and Jan Van Leeuwen. Vol. 2391. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 1–20. ISBN: 978-3-540-43928-8 978-3-540-45614-8. DOI: 10.1007/3-540-45614-7_1. URL: http://link.springer.com/10.1007/3-540-45614-7_1 (visited on 03/17/2026).

[6] Dan Suciu, Yisu Remy Wang, and Yihong Zhang. *Semantic foundations of equality saturation*. Jan. 5, 2025. DOI: 10.48550/arXiv.2501.02413. arXiv: 2501.02413[cs]. URL: http://arxiv.org/abs/2501.02413 (visited on 02/04/2026).

[7] Ashish Tiwari. *Decision procedures in automated deduction*. State University of New York at Stony Brook, 2000. URL: https://search.proquest.com/openview/30046982652b13cadc7b85d653cdf547/1?pq-origsite=gscholar&cbl=18750&diss=y (visited on 03/18/2026).

[8] Max Willsey et al. "egg: Fast and extensible equality saturation". In: *Proceedings of the ACM on Programming Languages* 5 (POPL Jan. 4, 2021), pp. 1–29. ISSN: 2475-1421. DOI: 10.1145/3434304. URL: https://dl.acm.org/doi/10.1145/3434304 (visited on 02/08/2026).

[9] Philip Zucker. *Omelets Need Onions: E-graphs Modulo Theories via Bottom-up E-matching*. Apr. 19, 2025. DOI: 10.48550/arXiv.2504.14340. arXiv: 2504.14340[cs]. URL: http://arxiv.org/abs/2504.14340 (visited on 01/20/2026).